Data Formats: XML & JSON

Week 4 · Foundations module · Textbook Chapter 4

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 9 & 11, 2026

This week at a glance

Two formats carry almost all web data: **XML** (a tree of tags) and **JSON** (nested lists and dictionaries). This week we learn to parse both.

Monday | Labor Day — no class

- Monday's concept intro is **folded into Wednesday**

Wednesday | Concepts & notebook lab

- XML & JSON concepts, then the Chapter 4 notebook: Congress, tweets, and weather

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 4

Companion notebook

`ch-04-data-formats.ipynb`

Short week

No class Monday (Labor Day). We meet **Wednesday** (concepts + notebook) and **Friday** (revisions).

1. Wednesday · Concepts & Notebook Lab

The role of data formats

Before you can *extract* data from the web, you have to understand how it is *structured*. Two formats dominate:

- **XML** — the older, document-centric format; a tree of nested tags.
- **JSON** — the newer, lighter format of the API economy; maps directly onto Python lists and dictionaries.

Every page, API response, and data feed you meet this semester is one of these — or HTML, which is XML's sibling.

When to use which

XML: older web services, government feeds (Congress, SEC, RSS), configuration files.

JSON: modern web APIs, database exports, service-to-service exchange.

A brief history of XML

XML began in 1996, when a W3C working group set out to simplify SGML — a much older, much more complicated markup standard — into something the web could actually use.

The goal was explicit: keep SGML's extensibility (define your own tags) while making documents both human-readable *and* machine-parseable, without SGML's complexity.

XML 1.0 became a W3C Recommendation on **February 10, 1998**. It was designed as a foundation, not a single format — a whole family of languages is built on it: XHTML, SVG, MathML, RSS, RDF.

Timeline

1996 — W3C working group begins

Feb 1998 — XML 1.0 becomes a W3C Recommendation

Today — backbone of RSS, SOAP, government feeds, Office file formats

A brief history of JSON

Douglas Crockford specified JSON in the early 2000s — describing a format he noticed was already implicit in JavaScript's own object syntax, rather than inventing something new. `json.org` went live not long after.

JSON's design goal was the opposite of XML's: no tags, no schema, no comments — just the two structures (objects and arrays) that "virtually all modern programming languages support," built to be parsed by a machine in one step.

Standardization came later, in stages: RFC 4627 (2006), ECMA-404 (2013), and the current IETF standard, RFC 8259 (2017). By the 2010s, AJAX and the API economy had made it the default.

Timeline

Early 2000s — Crockford specifies JSON; `json.org` launches

2006 — RFC 4627, the first formal spec

2013 / 2017 — ECMA-404, then RFC 8259

XML: a tree of nested tags

XML organizes data as a **hierarchy**: each tag can hold text, attributes, and child tags. The House `MemberData` feed, drawn as a tree:

```
<MemberData>
  <member> # repeats 441x
    <member-info>
      <firstname>Joe</firstname> # value in TEXT
      <lastname>Neguse</lastname>
      <party>D</party>
    </member-info>
    <state postal-code="CO"> # value in an ATTRIBUTE
      <state-fullname>Colorado</state-fullname>
    </state>
  </member>
</MemberData>
```

A value hides in one of two places: **text** (read with `.text`) or an **attribute** (read with brackets).

XML and HTML: siblings, not parent and child

XML and HTML are often assumed to be one a subset of the other. They are not — they are **siblings**. Both descend from SGML, an older and far more complex markup standard from the 1980s.

The difference is strictness. XML enforces well-formedness: every tag closes, attributes are quoted, nesting is exact — break any rule and the parser refuses to continue. HTML historically did not: browsers were built to guess at malformed markup and render it anyway.

Only the stricter **XHTML** dialect is actually valid XML. The HTML you scrape in the wild almost never is — which is exactly why Ch. 6 teaches you a forgiving parser instead of a strict one.

The XML family

XHTML, SVG, MathML, RSS, and RDF are all XML — "define your own tags" turns out to be useful for more than one problem.

JSON: lists and dictionaries you already know

JSON maps onto two Python structures you already use. The weather forecast, peeled one layer at a time:

```
{ # dict: type(data) is dict
  "latitude": 40.01,
  "daily": { # dict: data["daily"]
    "time": [...], # list: data["daily"]["time"]
    "temperature_2m_max": [...]
  }
}
```

A JSON **array** is a Python **list**; a JSON **object** is a Python **dict**. Real data nests them several levels deep — the skill is navigating from the outside in, one `type()` and `.keys()` call at a time.

JSON vs. XML: pick your trade-offs

JSON

+ Compact — no repeated tag names
+ Parses natively in JavaScript
+ Simple: objects, arrays, 4 scalar types
– No comments allowed
– No native schema enforcement
– No display semantics — data only

XML

+ Schema validation (DTD/XSD) before parsing
+ Rich vocabulary: attributes, mixed content, namespaces
+ Can double as a document format (SVG, DOCX)
– Verbose — every tag closes
– No native browser/JS parsing
– Everything parses as text; you convert types

Neither is “better” — each is a different bet about who, or what, reads the document.

Where JSON shows up

JSON dominates anywhere a **program** is the primary reader.

REST APIs return it almost by default now. SOAP, the XML-based protocol REST displaced, wraps every request in an XML namespace envelope that JSON has no equivalent for — part of why SOAP use has declined for years while REST and GraphQL grew.

JSON Web Tokens (the credential your browser holds after you log in) are JSON underneath. So are most modern config files — `package.json`, Kubernetes manifests — because the ecosystem around them is JavaScript-adjacent and a program on both ends is doing the reading.

Rule of thumb

If a program wrote it and a program will read it, expect JSON.

Where XML persists

XML persists where a **human** sometimes needs to read the document, where a **strict contract** matters, or where the data **predates the JSON era**.

Government and legislative systems (today's House roster), SEC filings, and enterprise data interchange still commonly use XML — partly legacy, partly because XSD validation mechanically guarantees a submitted document has the fields it is supposed to have.

And XML hides in plain sight: open a `.docx`, `.xlsx`, or `.svg` file's internals and you are looking at it. Office files are a zip archive of XML documents; SVG is XML a browser renders directly.

Rule of thumb

If a document needs to double as data *and* something a human might open, expect XML.

One pipeline unifies the whole course

Whether data starts as XML, JSON, HTML, or PDF text, the move into analysis is always the same:

find records → **extract fields** →
list of dictionaries → `pd.DataFrame`

The DataFrame is the **universal intermediate format**. Reach it and all of pandas opens up: filter, group, cross-tabulate, merge.

The source changes; the tag names change; **the pipeline stays the same**.

You reuse this pattern in…

- HTML tables — Ch. 6
- archived pages — Ch. 7
- PDF text — Ch. 9
- Wikipedia, government & social APIs — Ch. 10–12

Parsing XML with BeautifulSoup

Parse the string into a navigable tree, then search it. `.find()` returns the first match, `.find_all()` returns them all — each searches the *entire* subtree beneath a tag:

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(house_raw, "xml") # the "xml" parser comes from lxml

members = soup.find_all("member")
print(len(members)) # 2

for m in members:
    first = m.find("firstname").text # .text = content between the tags
    last = m.find("lastname").text # .find() reaches nested tags too
    party = m.find("party").text
    print(f"{first} {last} ({party})") # Joe Neguse (D) / Jeff Hurd (R)
```

The two-place problem — and the crash that guards it

A value is either **text** or an **attribute**:

```
state = soup.find("state")
state["postal-code"] # "CO"
state.find("state-fullname").text # "Colorado"
```

A missing tag makes `.find()` return `None`:

```
# AttributeError: None has no .text
m.find("party").text
# guard every extraction instead:
m.find("party").text if m.find("party") else None
```

Working out *which* of the two holds your value is half the parsing battle — real XML uses both.

Calling `.text` on a `None` is the **single most common** scraping error. The `if ...else None` guard keeps one missing field from crashing a 441-record loop.

XML always hands you **text**: converting `"2nd"` or `"$1,234"` to a number is your job.

XML → DataFrame: the real House roster

```
import requests, pandas as pd

url = "https://clerk.house.gov/xml/lists/MemberData.xml"
soup = BeautifulSoup(requests.get(url).text, "xml")
print(soup.find("MemberData")["publish-date"])

records = []
for m in soup.find_all("member"): # 441 members
    st = m.find("state")
    records.append({"last_name": m.find("lastname").text,
                    "party": m.find("party").text,
                    "state": st["postal-code"] if st else None})
df = pd.DataFrame(records) # find_all -> dicts -> DataFrame
```

The raw response, before parsing:

```
July 1, 2026
<member>
 <member-info>
  <firstname>Joe</firstname>
  <lastname>Neguse</lastname>
  <party>D</party>
 </member-info>
 <state postal-code="CO">
  <state-fullname>
   Colorado
  </state-fullname>
 </state>
</member>
```

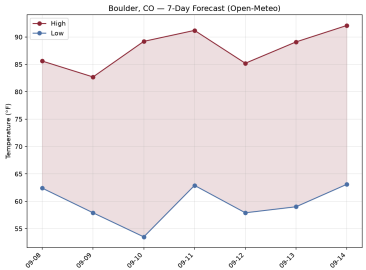
JSON: parse a string, then the list-of-dictionaries pipeline

`json.loads()` turns a string into Python objects (`json.load()` reads a file). With `requests`, `response.json()` does it for you:

```
import json, pandas as pd

data = json.loads('{"state": "Colorado", "population": 5773714}')
print(data["state"]) # Colorado
# response.json() == json.loads(response.text)

census = [ # the list-of-dictionaries pattern:
    {"state": "Colorado", "year": 2020, "population": 5773714},
    {"state": "Colorado", "year": 2010, "population": 5029196},
]
df = pd.DataFrame(census) # one dict per row, keys -> columns
```

JSON: peel the onion on a live API

```python
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {"latitude": 40.01, "longitude": -105.27,
          "daily": "temperature_2m_max",
          "timezone": "America/Denver"}
data = requests.get(url, params=params).json()

# type() + .keys(), one level deeper each time
data.keys() # latitude, longitude, daily...
data["daily"].keys() # time, temperature_2m_max
highs = data["daily"]["temperature_2m_max"] # a list
data.get("elevation", "unknown") # .get avoids KeyError
```



Boulder's 7-day forecast, fetched live on Sept. 8.

Lab: work these in pairs, then show-and-tell

Work in pairs. Do these exercises from the Chapter 4 notebook:

1. **RSS feed** — parse a news feed with the `"xml"` parser. Extract the title, the link, and the date. Put them in a DataFrame.
2. **Nested JSON** — get the dates and the maximum temperatures from Open-Meteo. Put them in a DataFrame.
3. **XML and JSON** — get the same data in both formats. Report which format is easier to parse. Report which format has more metadata.
4. **Reusable** — write `xml_to_dataframe(soup, tag, fields)`. Test it on the Congress data. Then test it on an RSS feed.
5. **Weather** — plot the high and low temperatures. Mark the days below freezing.
6. **5617**: write a critique of a public XML or JSON dataset. Use the datasheet format of Gebru et al. (2021)

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

Error flavors

Some code here is *designed* to break (a guard demonstrating a crash) — that does not need a GitHub issue. Other code may be *genuinely* broken: confirm it, then search and file an issue.

Effort over perfection

Start the lab in class, finish at home, submit what you got working. Graded on a good-faith attempt to complete the notebook, not on getting every cell to run.

Submitting

Run all cells, then export: *File → Save and Export Notebook As → HTML*. Upload to Canvas by **Sunday 11:59pm**.

The data source changes.
The tag names change.
The pipeline stays the same.

`find_all` → dicts → `DataFrame`.

2. Friday · Textbook Revisions

Friday — improving Chapter 4 together

Today we make the book better. Each of you opens a **pull request** against `ch-04-data-formats.qmd` and reviews a classmate's.

The workflow (from Week 3):

1. Fork or branch the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

The reviewer's tool: a suggestion

````
```suggestion
XML 1.0 became a W3C
Recommendation in 1998.
```
````

One click for the author to accept it.

A revision menu for Chapter 4

High-value targets — pick one and scope it to a single PR:

- **Test the code against live sources.** The House roster drifts (441 today; the `publish-date` changes) and Open-Meteo dates roll forward. Run the notebook, report what changed, and pin the snapshot.
- **Close a gap in "Common Issues to Debug."** Add a worked `json.JSONDecodeError` case — a trailing comma, single quotes, or an HTML error page returned instead of JSON.
- **Verify a real-world-patterns claim.** The chapter names specific places JSON and XML show up (JWTs, SOAP, Office file formats). Check one against a primary source and tighten the citation.
- **Strengthen an exercise.** Give the RSS or weather exercise expected output, a rubric, or a starter cell so it is self-checking.
- **Extend the RSS / post-API story.** Tie the decline-of-RSS narrative to a concrete citation and the "soft enclosure" theme (Ch. 3).

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Renders (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Web Architecture & Protocols** — how a URL becomes a rendered page (HTTP, requests, status codes).

Key takeaways

1. **XML** is a tree of tags; **JSON** is nested lists and dictionaries — the two structural languages of web data.
2. XML (1998, from SGML) favors strict contracts and documents a human might read; JSON (early 2000s, from JavaScript) favors compact, program-to-program exchange.
3. BeautifulSoup navigates XML with `.find()`, `.find_all()`, and `.text`; attributes come out with bracket notation. For JSON, **peel the onion** — `type()` and `.keys()`, top-down.
4. The `find_all` → list-of-dicts → `DataFrame` pipeline is the bridge from raw web data to analysis.
5. Format is a **transparency** choice: open XML and JSON support public oversight; scanned PDFs prevent it.